

Genetic Fuzzy Tree (GFT) for N-CMAPSS

Code documentation

`gft.py · datasets.py · leaf_search.py · ncmappss_features.py · gft_analysis.py · run.py`

1 Overview

The model is a *nested* Genetic Fuzzy Tree trained **end-to-end** by a real-coded genetic algorithm. It is a genuine tree, not a chain of independent stages: small sub-FIS are composed, and the *output of a child FIS is an input of its parent*.

- **Leaves** — **diagnosis**. One FIS per health modifier maps residualised cruise sensors to an estimated health parameter, `[sensors] -> theta_hat`. Leaves are *supervised*: each has a θ target.
- **Spool nodes** — **aggregation**. `hp` and `lp` fuse leaf outputs into one *latent* shaft-damage value in $[0, 1]$. No target of their own.
- **Root** — **prognosis**. `RUL(hp, lp)` outputs remaining useful life.

The whole genome is optimised against a single scalar loss; the latent spool values are discovered by the GA.

The claim being defended. The purpose is *not* to beat a recurrent network on RMSE. It is to obtain a prognostic model whose intermediate nodes are physically named and readable, and which generalises to **unseen failure modes** better than a lifetime prior. Every design decision below is subordinate to that, and several trade accuracy for it deliberately.

Fuzzy model. Each FIS is a **zero-order Takagi–Sugeno** system on a *full grid* of rules. Inputs are fuzzified with a **Ruspini partition** (triangles summing to 1 everywhere). With the product t-norm and the full grid, the firing strengths sum to exactly 1,

$$\sum_{\text{rules}} \prod_d \mu_{d,i_d}(x_d) = \prod_d \left(\sum_i \mu_{d,i}(x_d) \right) = 1,$$

so the Sugeno weighted average collapses to a dot product with the rule singletons:

$$y(x) = \sum_r f_r(x) c_r.$$

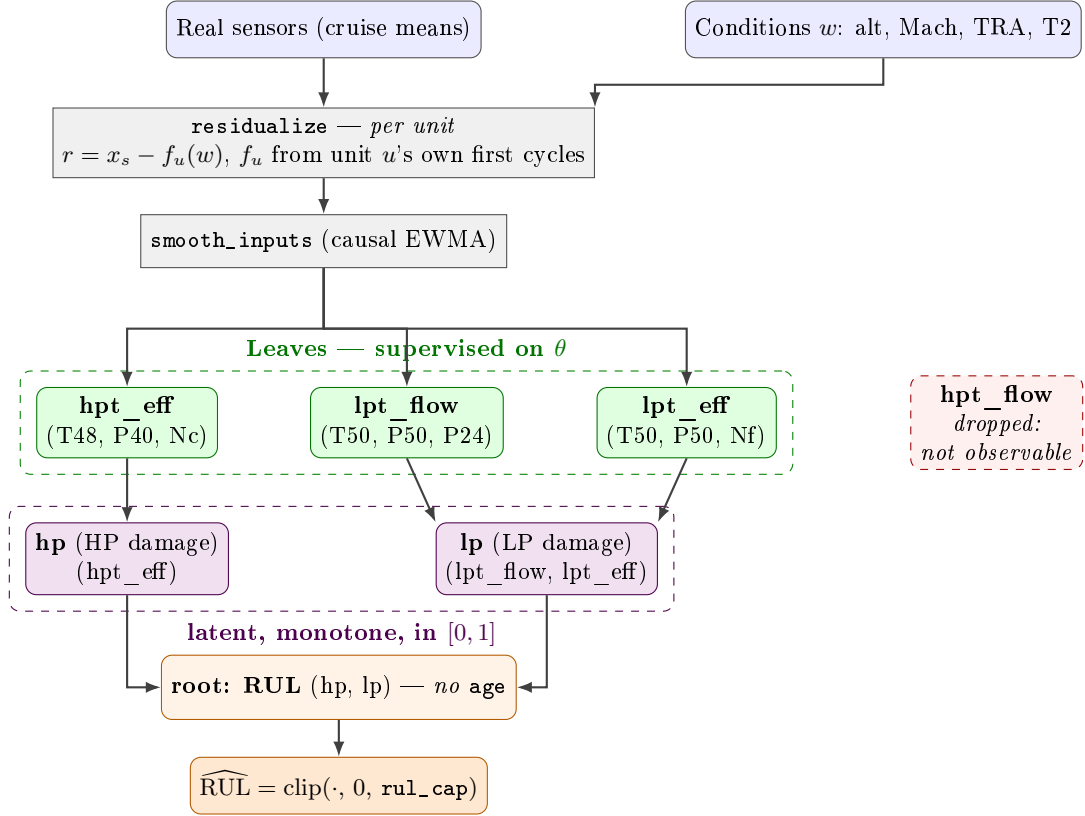
The genome holds *both* halves of the fuzzy system. A genetic fuzzy system that tunes only the consequents is half a GFS. Here the genome carries the **antecedents** (where the membership functions sit, §3) and the **consequents** (one singleton per rule, §4).

The tree must beat one line of arithmetic. Every result is reported next to

$$\widehat{\text{RUL}} = c - \text{age}, \quad c = \text{one fitted constant},$$

which uses *zero sensors*. On units with similar lifetimes this baseline is strong, and a GA given **age** at the root will reproduce it while the leaves become decoration. Hence §7.

2 Tree architecture



Each engine's sensors are stripped of *its own* new-condition signature (§5), EWMA-smoothed along cycles, then fed to the leaves. **hp** is a **single-input node**: a monotone 1-D calibration from HPT efficiency modifier to HP-shaft damage. That is intentional — see §8. A one-input FIS is still a legitimate, readable fuzzy node, and `plot_surface_node` renders it as a curve.

Three tree variants live in `gft.py`:

constant	params	role
TREE	158	default
TREE_LP1	109	drops the coupled <code>lpt_eff</code> ; fully spool-specific
TREE_AGE	180	ablation control — restores age at the root

Editing the topology. A tree is a list of (name, [inputs], theta_target_or_None). An input is a data column (sensor or "age") or the name of an earlier node. Children must precede parents, the root must be named "RUL", and a node takes at most 3 inputs. Per-input membership counts are given by writing an input as a pair:

```

_LEAVES = [
    ("hpt_eff", [("T48", 5), "P40", "Nc"], "HPT_eff_mod"), # T48 -> 5 fuzzy sets
    ("lpt_flow", ["T50", "P50", "P24"], "LPT_flow_mod"),
    ("lpt_eff", ["T50", "P50", "Nf"], "LPT_eff_mod"),
    ("hp", ["hpt_eff"], None), # single-input spool node
    ("lp", ["lpt_flow", "lpt_eff"], None),
]
TREE = _LEAVES + [("RUL", ["hp", "lp"], None)]
  
```

A node's rule count is the *product* of its inputs' set counts, so $(5, 3, 3) \Rightarrow 45$ rules and 45 consequent genes for that node alone.

3 Learnable membership functions

Why quantile centres fail. Fixing the antecedent centres at data quantiles is pathological for a health modifier: $\theta \approx 0$ for most of a unit's life and only plunges at the end, so the *median* sits at ≈ 0 . With $k = 3$, two of three centres collapse into the healthy plateau and the entire degraded regime — the only part that matters — is described by one triangle's ramp. It is calibrating an EGT gauge with tick marks at 0° , 1° and 900°C .

Centres *are* the membership functions. Under a Ruspini partition the fuzzy sets of one variable are entirely determined by their ordered centres $c_1 < \dots < c_k$. So the GA evolves them.

Encoding: gaps, not positions. A GA mutating positions directly would constantly emit out-of-order (invalid) partitions, needing a repair operator or a penalty. Instead each input contributes $k + 1$ strictly positive *gap* genes $g \in [\text{GAP_MIN}, 1]^{k+1}$, normalised and accumulated inside a padded domain $D = [d_{\text{lo}}, d_{\text{hi}}]$:

$$w = \frac{g}{\sum_j g_j}, \quad c_i = d_{\text{lo}} + (d_{\text{hi}} - d_{\text{lo}}) \sum_{j \leq i} w_j, \quad i = 1, \dots, k.$$

- **Ordering is structural.** $c_1 < \dots < c_k$ holds for *any* gene values, so no crossover or mutation can produce an invalid genome.
- **The partition stays Ruspini,** $\sum_i \mu_i(x) = 1$ everywhere, so the dot-product identity is preserved exactly. Self-test: $\max |\sum \mu - 1| = 0$.
- **Seeded at the previous model.** `_encode` inverts the map and the GA's first individual reproduces the quantile centres to $\sim 10^{-16}$, so learning MFs cannot *start* worse.
- **GAP_MIN** forbids two centres coinciding (a degenerate set).

Empirical status. Set `learn_mf=False` for the antecedent ablation. Across every run to date, MF learning has **not** shown a reliable held-out gain: in the most recent run the fixed-MF arm scored 9.34 ± 0.02 against 9.62 ± 1.26 for the learned arm — better mean and roughly fifty times more stable. The machinery is correct and produces the `plot_memberships` figure; whether to keep it in the final model is an open question.

4 Structural monotone consequents

A Sugeno grid is a generic approximator: nothing stops the singletons folding into a non-monotone surface, and on noisy data the GA *will* buy a fold if it shaves training loss. For the damage and RUL nodes that is unphysical — damage cannot fall with more damage — so monotonicity is baked into the parametrisation.

Encoding. A monotone node's rule slice stores [base | non-negative steps] (the same gene count as free singletons). The steps are cumulatively summed along every axis; axes whose sign is -1 are flipped before and after the cumsum so their running total grows the other way. **MONOTONE** gives the sign, and a **scalar broadcasts over any arity**:

```
MONOTONE = {"hp": -1, "lp": -1, "RUL": -1} # -1 = non-increasing in every input
```

Use a scalar rather than a tuple: it stays correct when a node's arity changes (the **TREE** root has 2 inputs, the **TREE_AGE** root has 3).

Signs are physics, not preference. `hp/lp` take damage *modifiers* (more negative = more degraded) and output shaft *damage*, which can only rise as a modifier falls. `RUL` takes damage in $[0, 1]$ (and, in **TREE_AGE**, *age*); more damage, or more accumulated cycles, can only mean less remaining life.

Leaves are deliberately unconstrained. Their monotonicity depends on the per-unit residual cleanly isolating the health effect, which is not guaranteed. A non-monotone leaf surface is therefore expected and is not a bug.

Range clipping is required, and safe. `base` is the grid **minimum** (the corner where the cumsum is zero), so the grid spans `[base, base+total]` — a box constraint can bound `base` and bound the total but cannot bound their sum. Without clipping, a spool grid runs past 1 and saturates against the `[0, 1]` clamp in `predict_tree`, wasting the node’s entire dynamic range. `_mono_decode` therefore clips to the node’s recorded output range; clipping is a monotone map, so monotonicity survives. Self-test: 500 random genomes, zero overshoot, zero violations.

Empirical status. `monotone=False` gives the ablation. An earlier configuration showed a clean win (8.87 vs 9.20 on validation, all three seeds, with much lower variance); after the loss and tree changed, the two arms are within noise. **Keep monotonicity for the interpretability claim; do not currently claim it improves accuracy.**

5 Preprocessing

Per-unit condition residual. Sensor readings are dominated by flight condition, not damage. For each unit u a quadratic condition model is fitted on that engine’s *own* first `ref_cycles` cycles and subtracted from all of its cycles:

$$r_u(t) = x_s(t) - [1, w(t), w(t)^2] \beta_u, \quad \beta_u = \arg \min_{\beta} \|X_u^{\text{first}} \beta - x_{s,u}^{\text{first}}\|^2.$$

So “T48 residual” means *hotter than this engine ran when it was new* — a damage signal. A single fleet-wide baseline instead means *hotter than the average engine ran when new*, i.e. damage + manufacturing scatter + flight-class bias; pooling many datasets multiplies that error term.

There is no leakage: it uses only early-life *sensor* data from the engine itself, available at deployment, and touches neither RUL nor the `hs` flag. Nothing is stored — the transform is recomputed identically on a held-out unit, so fit and predict cannot drift apart.

EWMA smoothing. The FIS is *memoryless*: one cycle’s cruise means in, one $\hat{\theta}$ out, no temporal context, so per-cycle sensor noise passes straight into the prediction. `smooth_span` EWMA-smooths the sensors within each unit (causal, hence no leakage).

6 Loss

Every term is expressed as a *fraction of a trivial baseline’s error*, so the weights are interpretable and a leaf’s variance no longer sets its influence:

$$\mathcal{L} = \frac{\text{NASA}(y, \widehat{\text{RUL}})}{\text{NASA}(y, \text{age-only})} + w_{\theta} \frac{1}{L} \sum_{\ell} \frac{\text{RMSE}(\hat{\theta}_{\ell}, \theta_{\ell})}{\text{std}(\theta_{\ell})} + w_{\text{tr}} \frac{1}{L} \sum_{\ell} [1 - \bar{\rho}_{\text{unit}}(\hat{\theta}_{\ell}, \theta_{\ell})],$$

with $y = \min(\text{RUL}, \text{rul_cap})$.

- **RUL term** divided by the NASA score of $\text{RUL} = c - \text{age}$, so a value below 1 literally means “beats the zero-sensor baseline”. The NASA score itself is the asymmetric PHM score: with $d = \hat{y} - y$, cost $\text{expm1}(-d/13)$ for $d < 0$ and $\text{expm1}(d/10)$ for $d \geq 0$, so *late* predictions — over-estimating remaining life, the dangerous direction — are penalised more steeply.
- **θ term** divided by each leaf’s *own* trivial error $\text{std}(\theta_{\ell}) = \text{RMSE}(\text{predict-the-mean})$. Dividing by *range* (the earlier version) let a near-constant leaf explode and drown the informative ones.

- **Correlation term** rewards *shape*. Runs repeatedly produced leaves with per-unit $\rho \approx 0.5$ – 0.6 but negative R^2 : they tracked degradation and got the scale wrong. RMSE punishes scale error and gives shape no credit; this term does. It is what a memoryless FIS on noisy sensors can actually deliver.

Why the RUL target is capped. Before degradation onset the engine is healthy, θ is flat, and RUL is determined by the unit’s total *lifetime*, not by any sensor: it is not learnable from the data the model is given. An uncapped target forces the loss to fit that unlearnable region — typically more than half the rows — and **age** is the only feature that can. `suggest_rul_cap` places the cap at the median RUL at onset, from the **hs** flag via `since_onset`.

Real sensors only. Only the measured sensors of C-MAPSS Table 2 are used. The virtual sensors (x_v) are never loaded: they are outputs of the same simulator that generated the θ targets (leakage risk) and do not exist on a real engine (they would break the deployability claim).

7 Evaluation protocol

The unit is the independent sample

Rows are (unit, cycle) cruise means. Unit 4 cycle 31 and cycle 32 are the same engine at the same damage state, differing by noise. A random 80/10/10 *row* split therefore measures interpolation on a curve already fitted, not prediction. **Every split is by unit** (`split_units`, `datasets.split`, `loo_cv`), with asserts that no unit appears in two sets.

Three splits, three claims

Split		Question	Function
Random 80/10/10, fixed by dataset	units, strati-	Can it predict a <i>new engine</i> with a fault mode it has seen?	<code>datasets.split</code>
The files’ dev/test groups	own	The number comparable with published N-CMAPSS results.	<code>datasets.official_split</code>
Leave-one- <i>dataset</i> -out		Can it read a <i>fault mode it has never seen</i> ?	<code>datasets.leave_one_dataset_out</code>

The last is the interesting one. Observed behaviour is consistent across every configuration tried: the tree wins hardest exactly where the age prior fails (datasets whose units have atypical lifetimes) and loses mildly where degradation is near-linear in cycle. That asymmetry is the result.

Model selection

`run.py` evaluates several arms on **validation**, then refits the winning arm on train+val and touches **test once**. Arms carry a **selectable** flag: the ablation controls (**free-grid**, which removes the monotonicity the interpretability claim rests on, and **+age**, which restores the shortcut the design exists to avoid) can win on RMSE without being eligible for the test slot. Hardcoding a config here is a real hazard — it once sent the worst and least stable arm to test.

Reporting θ

An RMSE of 8×10^{-4} on a variable whose range is 1.3×10^{-2} sounds tiny and means little — it is 6% of range, and the constant predictor already achieves $\approx \text{std}(\theta)$. **evaluate** therefore reports, per leaf, R^2 and the mean **per-unit Pearson correlation** ρ . For a memoryless FIS the plain RMSE is dominated by the per-cycle noise floor; ρ asks whether θ *moves with* the truth.

8 Leaf input selection

Leaf inputs are the *output* of `leaf_search.py`, not engineering judgement. Each candidate is fitted **standalone** (one FIS, `sensors` -> `theta`) and scored on **held-out units**. Two traps are avoided by construction:

1. Scoring uses the leaf’s **own** θ , never downstream RUL — which would reward inputs that help through the age-correlated path.
2. Fitting is standalone, not inside the tree, isolating “which sensors carry this θ ” from the spools, the root and the RUL loss.

Cross-spool specificity. A high ρ is not enough: **T48** appears in 80–100% of every target’s top ten, because it is a *global* damage proxy. **-confirm** therefore adds seed robustness (mean $\pm$ sd over N seeds) and

$$\text{specificity} = \rho(\text{own target}) - \rho(\text{same inputs} \rightarrow \text{opposite spool's target}).$$

Near zero means the combination reads global damage rather than that component. **-by-specificity** screens the *whole* candidate list on one seed and shortlists by specificity, so a genuinely specific combination buried down the ρ ranking still gets a full multi-seed hearing.

Result over all 286 three-sensor combinations of the 13 measured sensors.

leaf	inputs	ρ	specificity	verdict
<code>hpt_eff</code>	T48, P40, Nc	0.606	+0.071	keep — strongest, most stable
<code>lpt_flow</code>	T50, P50, P24	0.588	+0.376*	keep — best ρ and R^2
<code>lpt_eff</code>	T50, P50, Nf	0.549	−0.071	predictive, <i>not</i> spool-specific
<code>hpt_flow</code>	—	0.250	−0.340	dropped

* **Caveat:** the flow-pair specificity is inflated. Its foil (`HPT_flow`) is itself unpredictable, so `foil_rho` is automatically low for any `lpt_flow` candidate. Judge `lpt_flow` on ρ/R^2 . The *eff*-pair specificity is valid, since both targets reach $\rho \approx 0.6$.

Why `hpt_flow` was removed. Exhaustively: across all 286 combinations its ceiling is $\rho = 0.27$ while every other modifier reaches ≈ 0.63 , with negative held-out R^2 throughout and specificity -0.34 — candidate inputs predicted the *other* spool better than HPT flow itself. It is not observable from cruise-mean sensors by a memoryless FIS. Keeping it would have fed the tree a leaf that diagnoses nothing. `TREE_LP1` likewise drops `lpt_eff`, giving a fully spool-specific tree on 109 parameters.

9 Reference: `gft.py`

1. Fuzzy inference

`_centers(x, n)`

n ordered Ruspini centres at data quantiles. Seeds the GA, and is the fixed placement when `learn_mf=False`.

`_memberships(x, c)`

(N, k) triangular memberships; each row sums to 1, shoulders on the extreme sets.

`_fis(cols, centers, singletons)`

One FIS: product t-norm over the full rule grid, then the weighted average.

1b. Learnable membership functions (§3)

`_domain(lo, hi)`

Padded interval a variable's centres may occupy.

`_decode(inp, genes)`

k ordered centres from $k + 1$ positive gap genes.

`_encode(inp, c)`

Inverse; used once, to seed the GA at the quantile placement.

`GAP_MIN, MF_PAD`

Minimum gap gene (0.02); domain padding (5% of span).

2. Tree architecture

`TREE, TREE_LP1, TREE_AGE`

Default, spool-specific, and age-ablation trees.

`MONOTONE, _mono_signs(name, n_inputs, monotone)`

Per-node monotonicity signs; a scalar broadcasts over any arity.

`_parse_input, _leaf_sensors`

Input-spec parsing; the set of real sensor columns a tree consumes.

2b. Structural monotone consequents (§4)

`_mono_decode(base, steps, shape, signs, out=None)`

Monotone grid from a base plus non-negative steps, clipped to $\text{out} = (\text{lo}, \text{hi})$.

`_mono_encode(grid, shape, signs)`

Inverse, for seeding at a valid monotone start.

Tree construction and the genome

`build_tree(frame, tree, n_terms, learn_mf, monotone)`

Resolves inputs, seeds centres at quantiles, and hands each node its **slice** of the genome. Layout, per node in order: [gap genes of its inputs | its singletons]. An input's slice is `inp["mf"]`; a node's consequents are `node["rules"]`.

`centers(node, genome)`

The node's Ruspini centres, decoded from the genome.

`node_singletons(node, genome)`

The node's consequents; decodes and clips the monotone encoding where present.

`seed_genome(meta, lo, hi)`

GA seed: quantile centres, mid-box singletons.

`bake_centers(meta, genome)`

Writes learned centres into `meta` so `inspect` and the plots see the tuned partition.

`genome_bounds(meta, frame, rul_hi)`

Box for the genome; also records each node's output range as `out_lo/out_hi` for the monotone clip.

`predict_tree(frame, meta, genome)`

Evaluates the whole tree. Both centres and singletons are read out of the genome.

`n_params(meta), n_mf_params(meta)`

Total genes; how many are antecedents.

3–4. Losses and the genetic algorithm

`rmse(a, b), nasa_score(y_true, y_pred)`

RMSE; the asymmetric PHM score.

`_unit_corr(units, y, yhat)`

Mean per-unit Pearson correlation, nan-safe.

`genetic_optimize(loss, lo, hi, pop, gens, seed, x0, ...)`

Real-coded GA: 3-way tournament selection, BLX- α crossover, Gaussian mutation, two elites, cooling σ . Returns (`best`, `history`).

5. Preprocessing (§5)

`residualize(frame, sensors, conds, ref_cycles=20)`

Per-unit condition residual; stateless, recomputed identically on held-out units.

`smooth_inputs(frame, cols, span)`

Causal EWMA along cycles, within each unit.

`_prep(frame, tree, residual, ref_cycles, smooth_span)`

The preprocessing that fit and predict must apply identically.

6. Fit / predict

`fit_gft(frame, tree, n_terms, pop, gens, residual, ref_cycles, smooth_span, rul_cap, theta_weight, trend_weight, learn_mf, monotone, seed)`

Trains the whole genome end-to-end on the loss of §6. *Fits on frame* — score the result on a held-out frame; in-sample numbers are not evidence.

`predict_gft(frame, model)`

Full inference: `unit`, `cycle`, each `<theta>_hat`, each latent `<node>_h`, and `RUL_hat` clipped to `[0, rul_cap]`.

`inspect(model)`

Per FIS: inputs, target, grid, singleton range, monotone tag, and — when antecedents were learned — the tuned centres beside the quantile seed.

`disp(inp, c)`

Centres back in display units (raw sensor units for `col` inputs).

7. Held-out evaluation (§7)

`cap(y, rul_cap)`

The piecewise-linear target `min(RUL, cap)`.

`suggest_rul_cap(frame, q=0.5)`

RUL at degradation onset, median over units.

`split_units(frame, holdout)`

Splits by *unit*. Never by row.

`r2, unit_rho, baselines(train, test, rul_cap)`

R^2 ; mean per-unit ρ ; mean RUL and `RUL = c - age` — the numbers the tree must beat.

`evaluate(frame, model, name)`

Scores a fitted model on a held-out frame: RMSE, MAE, NASA, R^2 , and per leaf the θ R^2 and per-unit ρ .

`loo_cv(frame, tree, name, seeds, **fit_kw)`

Leave-one-unit-out CV over seeds.

`summarize(df)`

Mean $\pm$ sd across folds and seeds.

8. Self-test — python gft.py

Asserts three invariants, all of which must pass:

- over 100 random genomes the decoded centres are always ordered and $\max|\sum \mu - 1| = 0$;
- the GA seed reproduces the quantile centres to $\sim 10^{-16}$;
- over 200 random genomes the monotone nodes never violate their sign.

10 Reference: datasets.py

`verify(pattern)`

Opens every .h5 and reports OK/BROKEN with size and the HDF5 error. Run after any download: a truncated file (“eof = N, stored_eof = M”) is the classic failure. **Keep these files out of cloud-synced folders** — sync-on-demand can leave a partially materialised placeholder, and h5py seeks all over the file.

`probe(pattern)`

Per file: which θ modifiers it *actually* contains (read from `T_var`), unit counts, lifetime spread. Unreadable files are skipped.

`build_cache(pattern, cache, force)`

Reduces each .h5 to cycle-level cruise means and parks it as parquet, **one file at a time**: `load_raw` pulls a whole split into memory, so a multi-GB file is several GB while being reduced. Cached, the entire fleet is a few thousand rows.

`pooled(cache, include, theta, fill_missing_theta)`

Concatenates the cache. Unit IDs collide across files (each numbers from 1) and are remapped to $1000 \cdot (\text{file index} + 1) + \text{local unit}$; originals kept in `unit_local/ds`. A file that never degraded a component has that modifier filled with 0 — valid supervision.

`split(frame, fracs, seed, stratify)`

80/10/10 **by unit**, stratified within each dataset so every fault mode appears in all three sets; asserts no unit straddles. Small files round up to at least one val and one test unit.

`official_split(frame)`

The files’ own dev/test groups.

`leave_one_dataset_out(frame)`

Yields (`train`, `test`, `ds`) with one whole file held out.

11 Reference: leaf_search.py

`search_target(feats, target, k, all_sensors, ...)`

Ranks every k -subset of the candidate sensors for one θ target, scored on held-out units by per-unit ρ . `-all-sensors` removes the physical prefilter and searches all 13.

`confirm_target(feats, target, shortlist, seeds, ...)`

Re-evaluates a shortlist over several seeds and adds the cross-spool specificity; the engineering BASELINE is always included for comparison.

`_prescreen(feats, target, all_combos, n, min_rho)`

Cheap one-seed specificity screen over the *whole* candidate list, to choose the shortlist by specificity rather than by ρ .

CANDIDATES, BASELINE, FOIL

Physical prefilter per target; the engineering choice for reference; the opposite-spool target used as the specificity foil.

Note: combination membership is compared as **sets**, since `itertools.combinations` emits sensors in pool order — a string comparison silently never matches.

12 Reference: ncmappss_features.py

`load_raw(path, split)`

One .h5 split $\rightarrow$ per-timestamp (1 Hz) frame. Loads A, W, X_s, T only; the virtual sensors X_v are deliberately never read.

`cycle_features(path, split, cruise_frac, min_cruise)`

One row per (unit, cycle): cruise means of sensors and conditions, plus **age**, **since_onset** and RUL. θ is imposed once per flight and is constant within a cycle, so a cycle-level aggregate loses nothing.

SENSORS, THETA, CONDITIONS

The column groups.

13 Reference: gft_analysis.py

`metrics(feats, pred)`

RUL and per- θ error table, per unit and pooled.

`plot_rul / plot_nodes / plot_theta / plot_scatter`

Per-unit RUL trajectories; the latent hp_h/lp_h damage curves; $\hat{\theta}$ versus θ ; the RUL parity plot.

`plot_memberships(model)`

The GFS figure. One panel per FIS input: the *learned* Ruspini partition (solid) over the quantile partition it was seeded from (dashed grey). If the GA has dragged centres out of the healthy plateau towards the degraded tail, the partition is now spending its resolution where the signal is.

`plot_surface_node / plot_surfaces(model, kind)`

One control surface per sub-FIS with the learned centres marked; a **curve** for a single-input node, one panel per input pair otherwise.

`plot_fitness(model)`

GA convergence.

`analyze(feats, pred, model, surfaces, savedir)`

Runs the lot. **Pass the capped frame** — it compares `frame["RUL"]` to `RUL_hat`, so hand it `frame.assign(RUL=gft.cap(frame["RUL"], rul_cap))` or you plot a capped prediction against an uncapped truth.

14 Typical usage

```
# --- once: verify, probe fault modes, cache (slow, one .h5 at a time) -----
python datasets.py

# --- derive leaf inputs from evidence (slow) -----
python leaf_search.py --all-sensors
python leaf_search.py --confirm --by-specificity --topn 5

# --- the protocol -----
python run.py
```

or, programmatically:

```
import pandas as pd, gft, datasets as ds

feats = ds.pooled()
train, val, test = ds.split(feats, fracs=(0.8, 0.1, 0.1)) # BY UNIT

cap = gft.suggest_rul_cap(train) # from TRAIN only
FIT = dict(gens=400, pop=120, rul_cap=cap, smooth_span=7,
           theta_weight=1.0, trend_weight=1.0)

print(gft.baselines(train, val, cap)) # the number to beat
```

```

m = gft.fit_gft(train, gft.TREE, learn_mf=True, monotone=True, **FIT)
print(gft.evaluate(val, m, "GFT full"))

# ablations, each isolating one design decision
gft.evaluate(val, gft.fit_gft(train, gft.TREE_LP1, monotone=True, **FIT), "LP-specific")
gft.evaluate(val, gft.fit_gft(train, gft.TREE, monotone=False, **FIT), "free-grid")
gft.evaluate(val, gft.fit_gft(train, gft.TREE, learn_mf=False, **FIT), "fixed-MF")
gft.evaluate(val, gft.fit_gft(train, gft.TREE_AGE, monotone=True, **FIT), "+age")

# TEST is touched once, with the winning DEPLOYABLE arm
final = gft.fit_gft(pd.concat([train, val]), gft.TREE, **FIT)
print(gft.evaluate(test, final, "GFT (test)"))
gft.inspect(final)

```